UrbanResQ

Intelligent Urban Disaster Resource Allocation Platform

Document Type: System Architecture & Database Schema Reference

Version: 1.0

Database Engine: MySQL 8.0+ (InnoDB)

Backend: Node.js / Express / TypeScript

Frontend: React / TypeScript / Vite

Generated: August 2026

Table of Contents

1. System Overview
2. Technology Stack
3. Application Workflow Diagram
4. API Architecture
5. Allocation Engine Workflow
6. Database Schema — Entity Relationship Diagram
7. Table Definitions (Full Reference)
8. Stored Procedures
9. Views

1. System Overview

UrbanResQ is a full-stack decision-support platform for coordinating urban disaster response. It manages the complete lifecycle from disaster declaration through emergency request intake, intelligent resource allocation, rescue team dispatch, hospital/shelter management, and post-event analytics.

Core Capabilities

- **Real-time Dashboard** — live stats, charts, and activity feed for situational awareness
- **Command Center** — interactive disaster map with heatmaps, emergency queue, resource panel
- **Allocation Engine** — stored procedure that optimally matches resources to requests using priority scoring
- **What-If Simulation** — dry-run scenarios that don't affect production data
- **Citizen Reporting** — public-facing form with optional AI image assessment
- **Drone Surveillance** — live HUD overlay with detection bounding boxes
- **RBAC Security** — JWT tokens, 7 distinct roles, permission-based route guards

2. Technology Stack

Frontend
React 18, TypeScript, Vite, Tailwind CSS, Recharts, Leaflet Maps

Backend
Node.js, Express, TypeScript, JWT (jsonwebtoken), bcryptjs, Zod validation

Database
MySQL 8.0+ (InnoDB), stored procedures, views, triggers, computed columns

Real-time
Socket.IO for live dashboard updates and notifications

3. Application Workflow Diagram

End-to-end flow from disaster event through resolution.



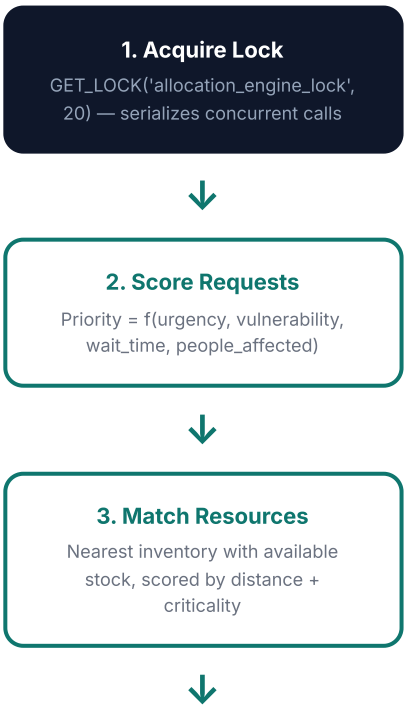
4. API Architecture

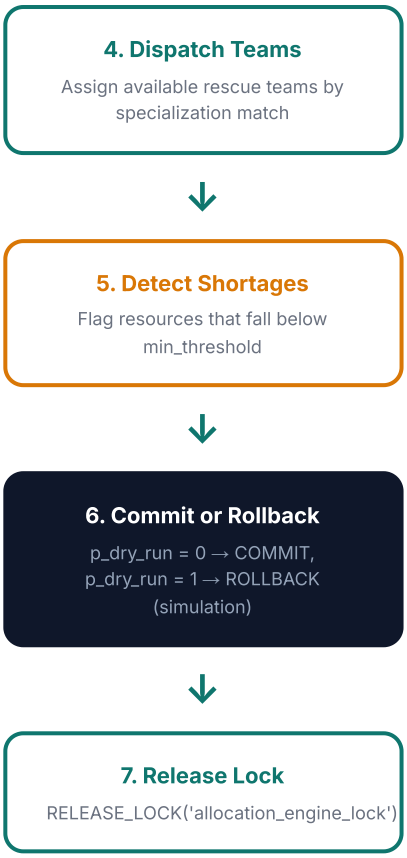
All API routes are mounted under `/api` with JWT-based authentication.

MOUNT POINT	METHODS	AUTH	DESCRIPTION
`/api/auth`	POST	Public	Login, register, token refresh
`/api/public`	POST	Public	Citizen emergency report (with image upload)
`/api/dashboard`	GET	Auth	Aggregated stats, trends, severity, activity
`/api/disasters`	CRUD	Auth + Role	Disaster lifecycle management
`/api/emergency-requests`	CRUD	Auth + Role	Request intake, status updates, priority ranking
`/api/allocations`	GET, POST	Auth + Role	View allocations, trigger engine run
`/api/resources`	GET	Auth	Available resources with utilization
`/api/inventory`	GET, POST	Auth + Role	Stock levels, manual adjustments
`/api/hospitals`	GET, POST	Auth + Role	Hospital beds, patient admissions
`/api/shelters`	GET, POST	Auth + Role	Shelter capacity, resident check-in
`/api/rescue-teams`	GET	Auth	Team availability and specialization
`/api/vehicles`	GET, POST	Auth + Role	Vehicle status, deployment to requests
`/api/volunteers`	GET, POST	Auth + Role	Volunteer assignment and release
`/api/simulations`	POST	Auth + Role	What-If scenario engine (dry run)
`/api/analytics`	GET	Auth	Response performance, trends, zone analysis
`/api/audit-logs`	GET	Auth + Admin	System-wide audit trail
`/api/search`	GET	Auth	Full-text search across entities

5. Allocation Engine Workflow

The allocation engine is a MySQL stored procedure (`sp_generate_resource_allocation_plan`) that serializes access via a named lock.




4. Dispatch Teams
Assign available rescue teams by specialization match
5. Detect Shortages
Flag resources that fall below min_threshold
6. Commit or Rollback
p_dry_run = 0 → COMMIT, p_dry_run = 1 → ROLLBACK (simulation)
7. Release Lock
RELEASE_LOCK('allocation_engine_lock')

6. Database Schema — Entity Relationship Diagram

Primary Key (PK) · Foreign Key (FK) · Data Type · Nullable / Optional

Identity & Access (4 tables)

USERS	
user_id	INT PK
full_name	VARCHAR(120)
email	VARCHAR(160) UQ
password_hash	VARCHAR(255)
phone	VARCHAR(20)
designation	VARCHAR(120)
is_active	TINYINT(1)
last_login_at	DATETIME

ROLES	
role_id	INT PK
role_name	VARCHAR(50) UQ
description	VARCHAR(255)

USER_ROLES	
user_id	FK → users
role_id	FK → roles
assigned_at	DATETIME

PERMISSIONS	
permission_id	INT PK
permission_key	VARCHAR(100) UQ
description	VARCHAR(255)

Disasters & Emergency Requests (6 tables)

DISASTERS	
disaster_id	INT PK
disaster_type	ENUM(10)
title	VARCHAR(180)
severity	ENUM(4)
status	ENUM(5)
affected_population	INT
latitude, longitude	DECIMAL(9,6)
metadata	JSON
created_by	FK → users

DISASTER_ZONES	
zone_id	INT PK
disaster_id	FK → disasters
location_id	FK → locations
zone_code	VARCHAR(20)
zone_name	VARCHAR(120)
severity	ENUM(4)

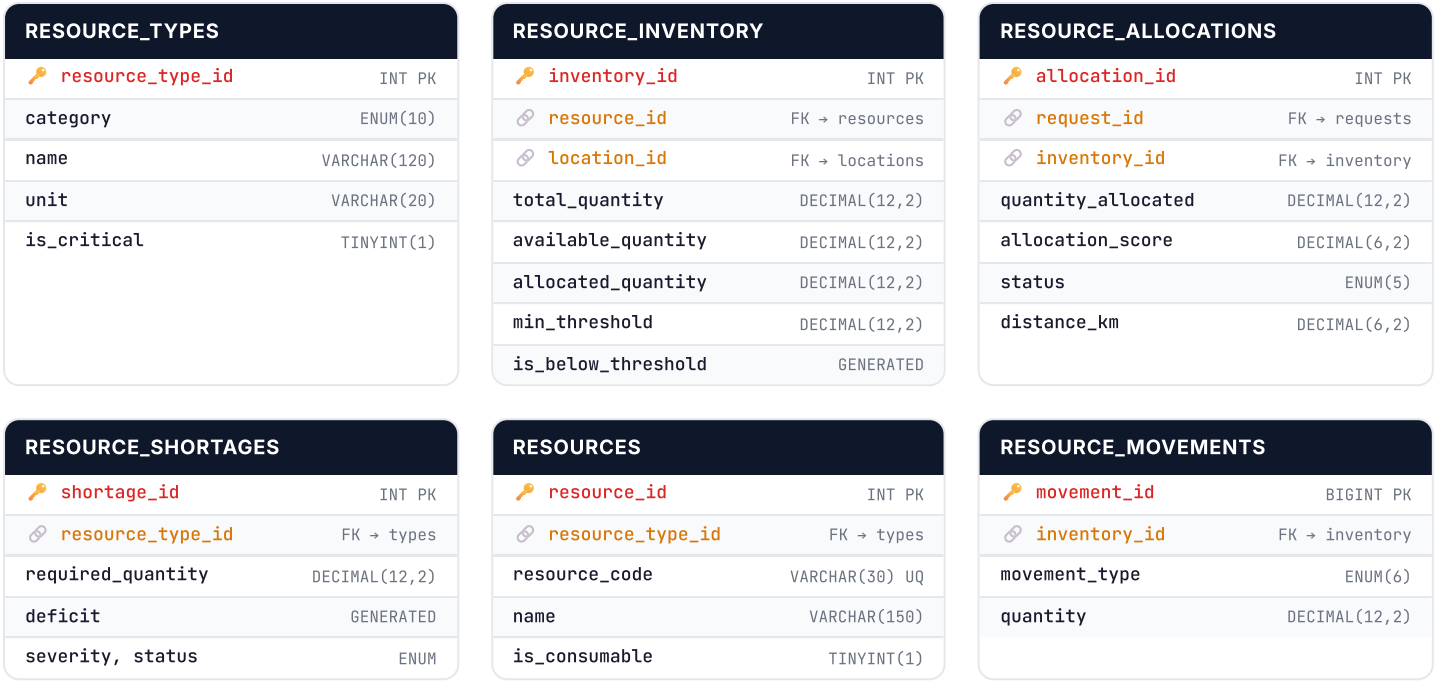
EMERGENCY_REQUESTS	
request_id	INT PK
disaster_id	FK → disasters
zone_id	FK → zones
location_id	FK → locations
category	ENUM(9)
urgency	ENUM(4)
status	ENUM(6)
priority_score	DECIMAL(6,2)
people_affected	INT
ai_assessed	BOOLEAN

CITIZENS	
citizen_id	INT PK
full_name	VARCHAR(120)
phone	VARCHAR(20)
vulnerable_group	ENUM(6)

INCIDENTS	
incident_id	INT PK
disaster_id	FK → disasters
incident_type	VARCHAR(100)
severity, status	ENUM

LOCATIONS	
location_id	INT PK
label	VARCHAR(150)
zone_name, city	VARCHAR
latitude, longitude	DECIMAL(9,6)

Resources & Allocations (6 tables)


RESOURCE_TYPES
resource_type_id INT PK
category ENUM(10)
name VARCHAR(120)
unit VARCHAR(20)
is_critical TINYINT(1)
RESOURCE_INVENTORY
inventory_id INT PK
resource_id FK → resources
location_id FK → locations
total_quantity DECIMAL(12,2)
available_quantity DECIMAL(12,2)
allocated_quantity DECIMAL(12,2)
min_threshold DECIMAL(12,2)
is_below_threshold GENERATED
RESOURCE_ALLOCATIONS
allocation_id INT PK
request_id FK → requests
inventory_id FK → inventory
quantity_allocated DECIMAL(12,2)
allocation_score DECIMAL(6,2)
status ENUM(5)
distance_km DECIMAL(6,2)
RESOURCE_SHORTAGES
shortage_id INT PK
resource_type_id FK → types
required_quantity DECIMAL(12,2)
deficit GENERATED
severity, status ENUM
RESOURCES
resource_id INT PK
resource_type_id FK → types
resource_code VARCHAR(30) UQ
name VARCHAR(150)
is_consumable TINYINT(1)
RESOURCE_MOVEMENTS
movement_id BIGINT PK
inventory_id FK → inventory
movement_type ENUM(6)
quantity DECIMAL(12,2)

Rescue Teams & Vehicles (5 tables)

RESCUE_TEAMS	
team_id	INT PK
team_code	VARCHAR(20) UQ
team_name	VARCHAR(120)
specialization	ENUM(7)
availability	ENUM(3)
current_location_id	FK → locations

VEHICLES	
vehicle_id	INT PK
vehicle_code	VARCHAR(20) UQ
vehicle_type	ENUM(6)
team_id	FK → teams
status	ENUM(3)

PERSONNEL	
personnel_id	INT PK
full_name	VARCHAR(120)
role	VARCHAR(80)

TEAM_MEMBERS	
team_id	FK → teams
personnel_id	FK → personnel

VEHICLE_DEPLOYMENTS	
deployment_id	INT PK
vehicle_id	FK → vehicles
request_id	FK → requests
deployed_at	DATETIME

Hospitals & Shelters (6 tables)

HOSPITALS	
hospital_id	INT PK
name	VARCHAR(150)
location_id	FK → locations
total_beds / available_beds	INT
icu_beds / available_icu	INT
blood_supply_level	ENUM(4)

HOSPITAL_ADMISSIONS	
admission_id	INT PK
hospital_id	FK → hospitals
patient_name	VARCHAR(120)
condition_severity	ENUM(3)

SHELTERS	
shelter_id	INT PK
name	VARCHAR(150)
location_id	FK → locations
max_capacity / current_occupancy	INT
food_supply / water_supply	ENUM(4)

SHELTER_RESIDENTS	
resident_id	INT PK
shelter_id	FK → shelters
full_name	VARCHAR(120)
checked_in/out	DATETIME

HOSPITAL_BEDS	
bed_id	INT PK
hospital_id	FK → hospitals
bed_type	ENUM(3)
status	ENUM(3)

SHELTER_CAPACITY	
capacity_id	INT PK
shelter_id	FK → shelters
occupancy	INT
recorded_at	DATETIME

Suppliers, Volunteers & System (8 tables)

VOLUNTEERS	
volunteer_id	INT PK
full_name	VARCHAR(120)
skills	VARCHAR(255)

SUPPLIERS	
supplier_id	INT PK
name	VARCHAR(150)
category	ENUM(6)

ALERTS	
alert_id	INT PK
alert_type, severity	VARCHAR / ENUM
message	VARCHAR(255)
is_acknowledged	TINYINT(1)

AUDIT_LOGS	
audit_id	BIGINT PK
user_id	FK → users
action, entity_type	VARCHAR
old_value / new_value	JSON

NOTIFICATIONS	
notification_id	INT PK
user_id	FK → users
title, message	VARCHAR

SYSTEM_SETTINGS	
setting_key	VARCHAR(80) PK
setting_value	VARCHAR(255)

8. Stored Procedures

PROCEDURE	PARAMETERS	PURPOSE
`sp_generate_resource_allocation_plan`	`p_disaster_id INT, p_user_id INT, p_dry_run BOOLEAN`	Core allocation engine. Scores pending requests by priority, matches nearest available resources, dispatches rescue teams, detects shortages, and either commits or rolls back (simulation mode).
`sp_admit_patient`	`p_hospital_id, p_patient_name, p_condition, p_needs_icu, p_request_id`	Admits a patient to the nearest available bed, auto-selects ICU if needed, decrements available bed counts.
`sp_check_in_resident`	`p_shelter_id, p_full_name, p_citizen_id`	Registers a displaced resident at a shelter, increments occupancy, enforces capacity constraints.

9. Views

VIEW	PURPOSE
`vw_available_resources`	Joins resource_types → resources → resource_inventory → locations. Computes stock_status (NORMAL / WARNING / CRITICAL) based on threshold.
`vw_resource_utilization`	Calculates utilization_pct per resource type across all locations.
`vw_emergency_requests_live`	Enriches emergency_requests with zone_name, location_label, disaster_title, and live_priority_score (time-decayed).
`vw_disaster_summary`	Per-disaster aggregation: zone_count, total_requests, unresolved_requests, critical_requests, people_affected.